Panda's basics for data manipulation in Python

1. Understanding the Pandas Library 2

1.1 Series in Pandas 4

1.2 Data frame and operations 4

2.1 Creating Series using Pandas 5

2.2 Different Series Attributes 5

2.3 Series vs List 7

2.4 Series Operations 8

2.5 Series from CSV file 8

2.6 Different functions in Series 10

2.7 Different Algorithms in Series 11

2.8 Extracting Values from Series 16

2.9 Some key methods like .value counts ( ), .apply ( ) 17

1. Exploratory data analysis 18

3.1 Reading files 21

3.2 Data cleaning in python: Handling missing values and filling NA 21

3.3 Data preparation and pre-processing 21

3.4 Data feature engineering: handling categorical data 21

3.5 Data validation techniques in python 21

3.6 Data feature engineering: removing columns and rows from raw data 29

Module-3 Pandas basics for data manipulation in Python

1. Understanding the Pandas Library

Python Pandas is defined as an open-source library that provides high-performance data manipulation in Python. It is used for data analysis in Python and developed by Wes McKinney in 2008.

Pandas is built on top of the Numpy package, means Numpy is required for operating the Pandas. The data produced by Pandas are often used as input for plotting functions of Matplotlib, statistical analysis in SciPy, and machine learning algorithms in Scikit-learn.

Key Features of Pandas

1. **Handling of huge volume of data**

   The Pandas library offers a fast and efficient way to manage and explore data. It providing us key feature like **Series** and **DataFrames** that help us data manipulation and representation of data in various ways.

2. **Alignment and indexing**

   Labelling and Indexing are the key features that give a new formatting to our data. If we have data but we don't know where it belongs and what it tells us about. So we can use label features for labelling of data as per requirement.

3. **Handling missing data**

   As in real time, Data is very crude in nature. There are various source of data through which data is generated in real time. So there may be some missing data or value. Therefore, it is pertinent to handle the missing values properly so that they do not adulterate our results.

4. **Cleaning up data**

   It is true, Data can be very crude. So we may have the really messy data, so performing any analysis over such data would lead to severely wrong results. Thus it is very important that we clean our data up.

5. **Input and output tools**

   Pandas provide built-in tools for the purpose of reading and writing data. While analysing data, we need to read and write data into data structures, web service, databases, etc.

6. **Multiple file formats supported**

   In real time, Data is generating via various platform so we can be found in many different file formats. So pandas has the key libraries that can be used for data analysis and can read various file formats like JSON or CSV, Excel etc.

7. **Merging and joining of datasets**

   Data analysis is very key aspects in real world. While analysing data, we constantly need to merge and join multiple datasets to create a final dataset. This is important because if the datasets aren't merged or joined properly, then it is going to affect the results. Pandas can help to merge various datasets, with extreme efficiency so that we don't face any problems while analysing the data.

8. **Multiple features for Time Series**

   These features include the likes of moving window statistics and frequency conversion.

9. **Optimized performance**

   Pandas is said to have a really optimized performance, which makes it really fast and suitable for data science. The critical code for Pandas is written in C or Cython, which makes it extremely responsive and fast.

10. **Visualization of data**

    Visualizing the data is an important part of data science. It is what make the results of the study understandable by human eyes. Pandas have an in-built ability to help you plot your data and see the various kinds of graphs formed.

11. **Grouping of data**

    Pandas also has the ability to separate your data and grouping it according to the criteria you want. With the help of the features of Pandas like GroupBy, you can split data into categories of your choice, according to the criteria you set. The GroupBy function splits the data, implements a function and then combines the results.

12. **Masking data**

    When in a case certain data is not needed for analysis and thus it is important that you filter your data according to the things you want from it. Using the mask function in Pandas allows you exactly to do that. It is extremely useful since whenever it finds data which meets the criteria you set for elimination, it turns the data into a missing value.

13. **Data is unique**

    Data always has a lot of repetition, therefore it is important that you are able to analyse data which has only unique values. By using the function dataset.column.unique (), we can eliminate the data repetition.

14. **Perform mathematical operations on the data**

    The built-in function in Pandas allows you to implement a mathematical operation on the data. Sometimes the dataset you have, is just not of the correct order. So by using the mathematical operations we can arrange that data as per our criteria.

Pandas offers three types of data structures:

Structure de données	Dimensions	Description
Series	1	1D labelled homogeneous array, size immutable.
Data Frames	2	General 2D labelled, size-mutable tabular structure with potentially heterogeneously typed columns.

1.1 Series in Pandas

Series is a one-dimensional array that is capable of storing various data types. The axis labels are collectively called indexes.

(a)Creating a Series using list:

```
Import pandas as pd
a = [8, 9, 5]                # list
mydata = pd.Series(a)        # creation of series
print(mydata)
```

Output:

```
0    8
1    9
2    5
dtype: int64
```

(b)Creating series with specific labels:

```
import pandas as pd
a = [58, 68, 72]
myvar = pd.Series (a, index = ["a", "b", "c"])
print(myvar)
```

Output:

```
a    58
b    68
c    72
dtype: int64
```

(c) **Creating a series by using dictionary**

```
import pandas as pd
calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series (calories)
print (myvar)
```

Output:

```
day1    420
day2    380
day3    390
dtype: int64
```

1.2 Data frame and operations

A Data frame is a two-dimensional data structure. Data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.

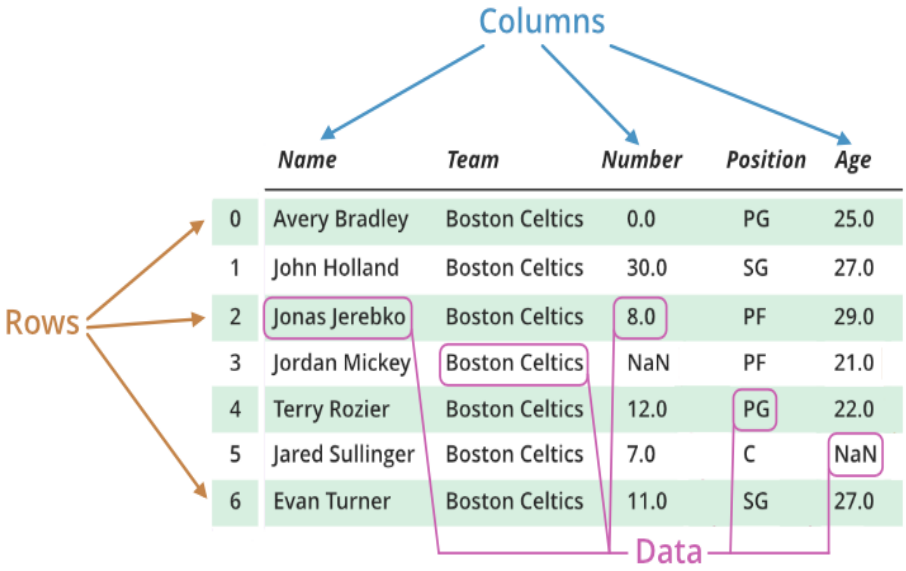
Creating Data Frame

Pandas DataFrame will be created by loading the datasets from existing SQL Database, CSV file, or an Excel file. DataFrame also can be created from lists, dictionaries etc. Data frame can be created by using the **DataFrame ( )** constructor.

```
import pandas as pd
df = pd.DataFrame()          # Calling DataFrame constructor
print (df)
data = ['BCA', 'MCA', 'BTech', 'MTech', 'MBA', 'PHD', 'Bsc']    # list of strings
df = pd.DataFrame(data)      # Calling DataFrame constructor on list
print (df)
```

Output

```
      0
0   BCA
1   MCA
2  BTech
3  MTech
4   MBA
5   PHD
6   Bsc
```



	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

2. Creating Series using Pandas

Series ( ) function is used to create a series in Pandas Library. We can create a series in different ways as below:

Creating an empty Series:

```
Import pandas as pd
ser = pd.Series()
print (ser)                    # By default, the data type of Series is float.
```

2.1 Different Series Attributes

We can Series by using the use different inputs like Numpy Array, Dict, and List etc. The Series attribute is used to define any information related to the Series object such as size, datatype. etc. Below are some of the attributes that we can use to get the information about the Series object:

Attributes	Description
Series.index	Defines the index of the Series.
Series.shape	It returns a tuple of shape of the data.
Series.dtype	It returns the data type of the data.
Series.size	It returns the size of the data.
Series.empty	It returns True if Series object is empty, otherwise returns false.

Series.hasnans	It returns True if there are any NaN values, otherwise returns false.
Series.nbytes	It returns the number of bytes in the data.
Series.ndim	It returns the number of dimensions in the data.
Series.itemsize	It returns the size of the datatype of item.

A. **Creating Series using Numpy array:**

We have to import the numpy module and then use array () function in the program. If the data is ndarray, then the passed index must be of the same length.

```
import pandas as pd
import numpy as np                    #import Numpy
info = np.array(['P','a','n','d','a','s'])    # array
a = pd.Series (info)
print(a)
```

```
0    P
1    a
2    n
3    d
4    a
5    s
```

B. **Create a Series from dictionary**

We can also create a Series from dictionary as mentioned below:

```
import pandas as pd
import numpy as np
info = {'x' : 0., 'y' : 1., 'z' : 2.}
a = pd.Series (info)
print (a)
```

```
x    0.0
y    1.0
z    2.0
dtype: float64
```

C. **Create a Series using Scalar:**

We can take the scalar values for creating a series but the index must be provided. The scalar value will be repeated for matching the length of the index.

```
import pandas as pd
import numpy as np
x = pd.Series (4, index= [0, 1, 2, 3])
print (x)
```

```
0    4
1    4
2    4
3    4
dtype: int64
```

D. **Accessing element of Series**

We can access the elements of the Series by using the following two methods:

- **Accessing Element from Series with Position**

  To access the element from a series, we need to use the index operator [ ]. The index must be an integer.

```
import pandas as pd
import numpy as np
```

Output

```
2    A
3    C
4    D
5    E
6    L
dtype: object
```

```
# creating simple array
data = np.array(['C','D','A','C','D','E', 'L','H','I','J','A','S','O','L','A'])
ser1 = pd.Series (data)
print (ser1[2:7])    # for retrieve the elements, slice method used
```

- **Accessing Element Using Label (index)**

  In order to access an element from series, we have to set values by index label. A Series is like a fixed-size dictionary in that you can get and set values by index label.

```
Import pandas as pd
Import numpy as np
data = np.array(['C','D','A','C','D','L', 'H','I'])
ser = pd.Series(data, index=[1,2,3,4,5,6,7,8])
print (ser[6])
```

Output:

L

2.2 Series vs List

There are the key attributes of python list and Series:

Python List	Python Series
A list is an ordered collection of elements, where each element has a specific index starting from 0.	Each element in the Series can be labelled with a unique index, which can be any hashable data type.
Lists are mutable, meaning you can add, remove, or modify elements after creating the list.	The size of a Series can be changed dynamically by adding or removing elements.
The elements in a list can be accessed by their index values.	Pandas Series supports vectorised operations, meaning mathematical operations can be performed on an entire Series.
The indexes of a list are always integers.	A series can store only similar kinds of data types to store the data.
A list can hold duplicate values.	Pandas Series can gracefully handle missing or NaN (Not a Number) values.
A list can store different types of data like string, int, float etc.	A Series can be sliced like a Numpy array using labels or integer positions.
	A scalar value can be broadcast to all the elements in the Series.
	Pandas Series can be created from a variety of data sources, including Numpy arrays, lists, and dictionaries.

2.3 Series Operations

There are following operations that we can perform on the list:

FUNCTION	DESCRIPTION
add()	Method is used to add series or list like objects with same length to the caller series
sub()	Method is used to subtract series or list like objects with same length from the caller series
mul()	Method is used to multiply series or list like objects with same length with the caller series
div()	Method is used to divide series or list like objects with same length by the caller series
sum()	Returns the sum of the values for the requested axis
prod()	Returns the product of the values for the requested axis
mean()	Returns the mean of the values for the requested axis
pow()	Method is used to put each element of passed series as exponential power of caller series and returned the results
abs()	Method is used to get the absolute numeric value of each element in Series/Data Frame
cov()	Method is used to find covariance of two series

A. add ( ): Python Series. Add () is used to add series or list like objects with same length to the caller series.

Syntax: Series. add (other, level=None, fill_value=None, axis=0)

Parameters:

Other: other series or list type to be added into caller series

fill_value: Value to be replaced by NaN in series/list before adding

Level: integer value of level in case of multi index

Return type: Caller series with added values

For Example:

```
import pandas as pd
 # Reading csv file from source
data = pd.read_csv('C:/Users/Lenovo/OneDrive/Desktop/data.csv')
data
short_data = data.head ()                    # creating short data of 5 rows
list =[50, 80, 90, 125, 205]                 # creating list with 5 values
# creating new column
short_data["Added values"]= short_data["Salary"].add(list)
# display
print(short_data.to_string())    // to print whole data_frame
```

B. Sub ( ):

Python is a great language for doing data analysis, primarily because of the fantastic ecosystem of data-centric Python packages. Pandas is one of those packages and makes importing and analysing data much easier.

Python **Series. Sub ()** is used to subtract series or list like objects with same length from the caller series.

Syntax: Series. Sub (other, level=none, fill_value=none, axis=0)

Parameters:

- Other: other series or list type to be subtracted from caller series
- fill_value: Value to be replaced by NaN in series/list before subtracting
- Level: integer value of level in case of multi index
- Return type: Caller series with subtracted values

```
Import pandas as pd
# Reading csv file from source
data = pd.read_csv('C:/Users/Lenovo/OneDrive/Desktop/data.csv')
data
short_data = data.head ()              # creating short data of 5 rows
list =[50, 80, 90, 125, 205]           # creating list with 5 values
# adding list data
# creating new column
short_data ["Added values"] = short_data ["Salary"].sub (list)
 # Display
print (short_data.to_string ())
```

2.4 Series from CSV file

Pandas series is a One-dimensional ndarray with axis labels. The labels need not be unique but must be a hashable type.

We can use **Series.from_csv()** function to read a csv file into a series. It is preferable to use the more powerful pandas.read_csv() for most general purposes.

Syntax: Series.from_csv (path, sep=', ', parse_dates=True, header=None, index_col=0, encoding=None, infer_datetime_format=False)

Parameter:

Path: string file path or file handle / StringIO

Sep: Field delimiter

parse_dates: Parse dates. Different default from read_table

Header: Row to use as header (skip prior rows)

index_col: Column to use for index

Encoding: a string representing the encoding to use if the contents are non-ascii

infer_datetime_format: If True and parse_dates is **True** for a column, try to infer the date time format based on the first date time string.

For Example:

```
Import pandas as pd
 # Reading csv file from source
data = pd.read_csv('C:/Users/Lenovo/OneDrive/Desktop/data.csv')
data
short_data = data.head ()                    # creating short data of 5 rows
list = [50, 80, 90, 125, 205]                # creating list with 5 values
# creating new column
short_data ["Added values"] = short_data ["Salary"].add (list)
# Display
print (short_data.to_string())    // to print whole data_frame
```

2.5 Different functions in Series

There are various functions in series that can be used for the various purpose.

FUNCTION	DESCRIPTION
Series()	A pandas Series can be created with the Series () constructor method. This constructor method accepts a variety of inputs
combine_first()	Method is used to combine two series into one
count()	Returns number of non-NA/null observations in the Series
size()	Returns the number of elements in the underlying data
name()	Method allows to give a name to a Series object, i.e. to the column
is_unique()	Method returns Boolean if values in the object are unique
idxmax()	Method to extract the index positions of the highest values in a Series
idxmin()	Method to extract the index positions of the lowest values in a Series
sort_values()	Method is called on a Series to sort the values in ascending or descending order
sort_index()	Method is called on a pandas Series to sort it by the index instead of its values
head()	Method is used to return a specified number of rows from the beginning of a Series. The method returns a brand new Series
tail()	Method is used to return a specified number of rows from the end of a Series. The method returns a brand new Series
le()	Used to compare every element of Caller series with passed series.It returns True for every element which is Less than or Equal to the element in passed series

ne()	Used to compare every element of Caller series with passed series. It returns True for every element which is Not Equal to the element in passed series
ge()	Used to compare every element of Caller series with passed series. It returns True for every element which is Greater than or Equal to the element in passed series
eq()	Used to compare every element of Caller series with passed series. It returns True for every element which is Equal to the element in passed series
gt()	Used to compare two series and return Boolean value for every respective element
lt()	Used to compare two series and return Boolean value for every respective element
clip()	Used to clip value below and above to passed Least and Max value
clip_lower()	Used to clip values below a passed least value
clip_upper()	Used to clip values above a passed maximum value
astype()	Method is used to change data type of a series
tolist()	Method is used to convert a series to list
get()	Method is called on a Series to extract values from a Series. This is alternative syntax to the traditional bracket syntax
unique()	Pandas unique() is used to see the unique values in a particular column
nunique()	Pandas nunique() is used to get a count of unique values
value_counts()	Method to count the number of the times each unique value occurs in a Series
factorize()	Method helps to get the numeric representation of an array by identifying distinct values
map()	Method to tie together the values from one object to another
between()	Pandas between() method is used on series to check which values lie between first and second argument
apply()	Method is called and feeded a Python function as an argument to use the function on every Series value. This method is helpful for executing custom operations that are not included in pandas or numpy

Note: you can study in detail by using - https://www.geeksforgeeks.org/python-pandas-series/

2.6 Different Sorting Algorithms in Series

Sorting is defined as an arrangement of data in a certain order. Sorting techniques are used to arrange data in an ascending or descending order. It is a method used for the representation of data in a more comprehensible format. It is an important area of Computer Science. The efficiency of the algorithm is proportional to the number of items it is traversing. For a small amount of data, a complex sorting method may be more trouble than it is worth. On the other hand, for larger amounts of data, we want to increase the efficiency and speed as far as possible.

There are various real-life examples where we can use the sorting method:

Telephone Directory: It is a book that contains telephone numbers and addresses of people in alphabetical order.

Dictionary: It is a huge collection of words along with their meanings in alphabetical order.

Contact List: It is a list of contact numbers of people in alphabetical order on a mobile phone.

The different types of sorting order are:

Increasing Order: A set of values are said to be increasing order when every successive element is greater than its previous element. For example: 1, 2, 3, 4, 5. Here, the given sequence is in increasing order.

Decreasing Order: A set of values are said to be in decreasing order when the successive element is always less than the previous one. For Example: 5, 4, 3, 2, 1. Here the given sequence is in decreasing order.

Non-Increasing Order: A set of values are said to be in non-increasing order if every ith element present in the sequence is greater than or equal to its (i-1)th element. This order occurs whenever there are numbers that are being repeated. For Example: 1, 2, 2, 3, 4, 5. Here 2 repeated two times.

Non-Decreasing Order: A set of values are said to be in non-decreasing order if every ith element present in the sequence is less than or equal to its (i-1)th element. This order occurs whenever there are numbers that are being repeated. For Example: 5, 4, 3, 2, 2, 1. Here 2 repeated two times.

Sorting Techniques: The different implementations of sorting techniques in Python are:

A. **Bubble Sort**
B. **Selection Sort**
C. **Insertion Sort**

A. **Bubble Sort**

Bubble Sort is a simple sorting algorithm. This sorting algorithm repeatedly compares two adjacent elements and swaps them if they are in the wrong order. It is also known as the sinking sort.

It has a time complexity of O(n2) in the average and worst cases scenarios and O(n) in the best-case scenario. Bubble sort can be visualized as a queue where people arrange themselves by swapping with each other so that they all can stand in ascending order of their heights.

We compare two adjacent elements and see if their order is wrong, if the order is wrong we swap them. (i.e arr[i] > arr[j] for 1 <= i < j <= s; where s is the size of the array, if array is to be in **ascending order**, and vice-versa).

For Example:

Here we sort the following sequence using bubble sort

Sequence: 2, 23, 10, 1

Step1: First Iteration

(2, 23, 10, 1) –> (2, 23, 10, 1), Here the first 2 elements are compared and remain the same because they are already in ascending order.

(2, 23, 10, 1) –> (2, 10, 23, 1), Here 2nd and 3rd elements are compared and swapped (10 is less than 23) according to ascending order.

(2, 10, 23, 1) –> (2, 10, 1, 23), Here 3rd and 4th elements are compared and swapped (1 is less than 23) according to ascending order

At the end of the first iteration, the largest element is at the rightmost position which is sorted correctly.

Step2: Second Iteration

(2, 10, 1, 23) –> (2, 10, 1, 23), Here again, the first 2 elements are compared and remain the same because they are already in ascending order.

(2, 10, 1, 23) –> (2, 1, 10, 23), Here 2nd and 3rd elements are compared and swapped (1 is less than 10) in ascending order.

At the end of the second iteration, the second largest element is at the adjacent position to the largest element.

Step3: Third Iteration

(2, 1, 10, 23) –> (1, 2, 10, 23), Here the first 2 elements are compared and swap according to ascending order.

The remaining elements are already sorted in the first and second Iterations. After the three iterations, the given array is sorted in ascending order. So the final result is 1, 2, 10, and 23.

```
# Python3 program for Bubble Sort Algorithm Implementation
def bubbleSort(arr):
    n = len(arr)
   # For loop to traverse through all
  # element in an array
  for i in range(n):
     for j in range(0, n-i-1):
         # Range of the array is from 0 to n-i-1
        # Swap the elements if the element found
```

```
        #is greater than the adjacent element
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
        # To print the fee in the sorting order
    fee = [ 200, 100, 150, 230 ]
    bubbleSort(fee)
    print("Sorted array is:")
for i in range(len(fee)):
print ("%d" % fee[i])
```

- **Time Complexity: O(n2)**
- **Auxiliary Space: O(1)**

Complexity of Sorting Algorithms:

The efficiency of any sorting algorithm is determined by the time complexity and space complexity of the algorithm.

1. Time Complexity: Time complexity refers to the time taken by an algorithm to complete its execution with respect to the size of the input. It can be represented in different forms:

- **Big-O notation (O)**
- **Omega notation (Ω)**
- **Theta notation (Θ)**

2. Space Complexity: Space complexity refers to the total amount of memory used by the algorithm for a complete execution. It includes both the auxiliary memory and the input.

The auxiliary memory is the additional space occupied by the algorithm apart from the input data. Usually, auxiliary memory is considered for calculating the space complexity of an algorithm.

B. Selection Sort

This sorting technique repeatedly finds the minimum element and sort it in order. Bubble Sort does not occupy any extra memory space. During the execution of this algorithm, two subarrays are maintained, the subarray which is already sorted, and the remaining subarray which is unsorted. During the execution of Selection Sort for every iteration, the minimum element of the unsorted subarray is arranged in the sorted subarray. Selection Sort is a more efficient algorithm than bubble sort. Sort has a Time-Complexity of O(n2) in the average, worst, and in the best cases.

For Example

Here we sort the following sequence using the selection sort

Sequence: 7, 2, 1, 6

(7, 2, 1, 6) –> (1, 7, 2, 6), in the first traverse it finds the minimum element (i.e., 1) and it is placed at 1st position.

(1, 7, 2, 6) –> (1, 2, 7, 6), in the second traverse it finds the 2nd minimum element (i.e., 2) and it is placed at 2nd position.

(1, 2, 7, 6) –> (1, 2, 6, 7), in the third traverse it finds the next minimum element (i.e., 6) and it is placed at 3rd position.

After the above iterations, the final array is in sorted order, i.e., 1, 2, 6, 7.

Implementation of selection sort:

```
def selectionSort(array, size):
   for s in range(size):
      min_idx = s
         for i in range(s + 1, size):
          # For sorting in descending order
         # for minimum element in each loop
         if array[i] < array[min_idx]:
            min_idx = i
       # Arranging min at the correct position
      (array[s], array[min_idx]) = (array[min_idx], array[s])
data = [ 70, 32, 41, 56 ]
size = len(data)
selectionSort (data, size)
 print ('Sorted Array in Ascending Order is :')
print(data)
```

- **Time Complexity: O(n2)**
- **Auxiliary Space: O(1)**

C. Insertion Sort

This sorting algorithm maintains a sub-array that is always sorted. Values from the unsorted part of the array are placed at the correct position in the sorted part. It is more efficient in practice than other algorithms such as selection sort or bubble sort. Insertion Sort has a Time-Complexity of O(n2) in the average and worst case, and O(n) in the best case.

For Example: Here we sort the following sequence using the insertion sort

Sequence: 7, 2, 1, 6

Step1: (7, 2, 1, 6) –> (2, 7, 1, 6), In the first iteration, the first 2 elements are compared, here 2 is less than 7 so insert 2 before 7.

Step2:(2, 7, 1, 6) –> (2, 1, 7, 6), In the second iteration the 2nd and 3rd elements are compared, here 1 is less than 7 so insert 1 before 7.

Step3 :(2, 1, 7, 6) –> (1, 2, 7, 6), After the second iteration (1, 7) elements are not in ascending order so first these two elements are arranged. So, insert 1 before 2.

Step4 :(1, 2, 7, 6) –> (1, 2, 6, 7), During this iteration the last 2 elements are compared and swapped after all the previous elements are swapped.

Selection Sort algorithm in Python

```
def insertion_sort(list1):
 # Outer loop to traverse on len(list1)
     for i in range(1, len(list1)):
         a = list1[i]
          # Move elements of list1[0 to i-1],
        # which are greater to one position
        # ahead of their current position
        j = i - 1
              while j >= 0 and a < list1[j]:
           list1[j + 1] = list1[j]
           j -= 1
            list1[j + 1] = a
              return list1
list1 = [ 70, 22, 51, 46 ]
print("The unsorted list is:", list1)
print("The sorted new list is:", insertion_sort(list1))
```

- **Time Complexity: O(n2)**
- **Auxiliary Space: O(1)**

2.7 Extracting Values from Series

Series.str.extract ( ) function is used to extract capture groups in the regex pat as columns in a DataFrame.

Syntax: Series.str.extract (pat, flags=0, expand=True)

Parameter:

Pat: Regular expression pattern with capturing groups.

Flags: int, default 0 (no flags)

Expand: If True, return DataFrame with one column per capture group.

Returns: DataFrame or Series or Index

For Example:

```
# importing pandas as pd
import pandas as pd
 # Importing re for regular expressions
import re
 # Creating the Series
sr = pd.Series(['New_York', 'Lisbon', 'Tokyo', 'Paris', 'Munich'])
 # creating the index
indx = ['City 1', 'City 2', 'City 3', 'City 4', 'City 5']
 # set the index
sr.index = indx
 # Print the series
print(sr)
```

```
In [9]: runfile('C:/Users
City 1    New_York
City 2      Lisbon
City 3       Tokyo
City 4       Paris
City 5      Munich
dtype: object
```

2.8 Some key methods like Index. Value counts ( ), .apply ( )

Python is a great language for doing data analysis. Pandas **Index. Value_counts ( )** function returns object containing counts of unique values. The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Syntax: Index.value_counts (normalize=False, sort=True, ascending=False, bins=None, dropna=True)

Parameters:

normalize: If True then the object returned will contain the relative frequencies of the unique values.

sort: Sort by values

ascending: Sort in ascending order

bins: Rather than count values, group them into half-open bins, a convenience for pd.cut, only works with numeric data

dropna: Don't include counts of NaN.

Returns: counts: Series

For Example:

```
# importing pandas as pd
Import pandas as pd
# creating the index
idx = pd.Index(['Harry', 'Mike', 'Arther', 'Nick', 'Harry', 'Arther'], name ='Student')
# Print the Index
print(idx)
```

```
In [10]: runfile('C:/Users/Lenovo/.spyder-py3/temp.py', wdir='C:/Users/Lenovo/.spyder-py3')
Index(['Harry', 'Mike', 'Arther', 'Nick', 'Harry', 'Arther'], dtype='object', name='Student')
```

.apply ( ) method: Pandas. Apply allow the users to pass a function and apply it on every single value of the Pandas series.

Syntax: s.apply (func, convert_dtype=True, args=())

Parameters:

- func: .apply takes a function and applies it to all values of pandas series.
- convert_dtype: Convert dtype as per the function's operation.
- args=(): Additional arguments to pass to function instead of series.
- Return Type: Pandas Series after applied function/operation.

```
import pandas as pd
def calc_sum(x):
  return x.sum()
data = {
  "x": [50, 40, 30],
  "y": [300, 1112, 42]
}
df = pd.DataFrame(data)
x = df.apply (calc_sum)
print(x)
```

```
x     120
y    1454
dtype: int64
```

3. **Exploratory data analysis**

Exploratory Data Analysis (EDA) is an approach that is used to analyse the data and discover trends, patterns, or check assumptions in data with the help of statistical summaries and graphical representations. This allows you to get a better feel of your data and find useful patterns in it.

Types of EDA:

Depending on the number of columns we are analysing, we can divide EDA into two types.

A. **Univariate Analysis –** In univariate analysis, we analyse or deal with only one variable at a time. The analysis of univariate data is thus the simplest form of analysis since the

information deals with only one quantity that changes. It does not deal with causes or relationships and the main purpose of the analysis is to describe the data and find patterns that exist within it.

B. **Bi-Variate analysis –** This type of data involves two different variables. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship between the two variables.

C. **Multivariate Analysis** – When the data involves three or more variables, it is categorized under multivariate.

Depending on the type of analysis we can also subcategorize EDA into two parts.

- **Non-graphical Analysis** – In non-graphical analysis, we analyse data using statistical tools like mean median or mode or skewness
- **Graphical Analysis** – In graphical analysis, we use visualizations charts to visualize trends and patterns in the data.

Steps Involved in Exploratory Data Analysis:

Step 1: Import Python Libraries

Import all libraries which are required for our analysis, such as Data Loading, Statistical analysis, Visualizations, Data Transformations, Merge and Joins, etc.

- Pandas and Numpy have been used for Data Manipulation and numerical Calculations
- **Matplotlib and Seaborn** have been used for Data visualizations.

Step 2: Reading Dataset

The Pandas library offers a wide range of possibilities for loading data into the pandas DataFrame from files like JSON, .csv, .xlsx, .sql, .pickle, .html, .txt, images etc.

Analysing the Data

- The main goal of data understanding is to gain general insights about the data we can use **shape ( ) method** to get the rows and columns in the dataset.
- We can use **head ( )** method that will display only top 5 rows from the top position, while **tail ( )** method can be used for getting rows from bottom.
- **info()** helps to understand the data type and information about data, including the number of records in each column, data having null or not null, Data type, the memory usage of the dataset.
- **Check for Duplication:** we can use **nunique ()** for checking the duplicacy. Duplicated data can be handled or removed based on further analysis.

- **Missing Values Calculation: isnull**() is widely been in all pre-processing steps to identify null values in the data. **data.isnull().sum()** is used to get the number of missing records in each column

Step 3: Data Reduction

Some columns or variables can be dropped if they do not add value to our analysis. We can use **drop ( )** method.

Step 4: Data Cleaning/Wrangling

Some names of the variables are not relevant and not easy to understand. Some data may have data entry errors, and some variables may need data type conversion**.**

Step 5: Statistics Summary

The information gives a quick and simple description of the data. We can include Count, Mean, Standard Deviation, median, mode, minimum value, maximum value, range, standard deviation, etc. describe() function gives all statistics summary of data

Step 6: Data Transformation

```
Import pandas as pd
Import numpy as np
# read dataset using pandas
df = pd.read_csv('C:/Users/Lenovo/OneDrive/Desktop/detail.csv')
print(df)          # print whole data set
#print(df.head(5))   # print only top rows using head()
 # get the shape of the data
#df.shape
df.describe(include = 'all')
df.info()   # to get the information about the dataset
df.nunique()
#Handling Missing Values
df.isnull().sum()
#fill in the missing values
df["Gender"].fillna("No Gender", inplace = True)
 df.isnull().sum()
import seaborn as sns
import matplotlib.pyplot as plt
 sns.histplot(x='Salary', data=df, )
plt.show()
sns.boxplot( x="Salary", y='Team', data=df, )
plt.show()
# Scatter Boxplot For Data Visualization
sns.scatterplot( x="Salary", y='Team', data=df,  hue='Gender', size='Bonus %')
# Placing Legend outside the Figure
plt.legend(bbox_to_anchor=(1, 1), loc=2)
 plt.show()
sns.pairplot(df, hue='Gender', height=2)
```

3.1 Reading files

The Pandas library offers a wide range of possibilities for loading data into the pandas DataFrame from files like JSON, .csv, .xlsx, .sql, .pickle, .html, .txt, images etc.

We can use read_csv( ) method for reading the datset.

Syntax: df = pd.read_csv ('C:/Users/Lenovo/OneDrive/Desktop/detail.csv')

3.2 Data cleaning in python: Handling missing values and filling NA

Data cleaning refers to the process of removing unwanted variables and values from your dataset. It may be removing missing values, outliers, and unnecessary rows/ columns. Re-indexing and reformatting our data. We can use **data.isnull ( ).sum ( )** to get the number of missing records in each column.

3.3 Data preparation and pre-processing

Data pre-processing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model.

Why do we need Data Pre-processing?

A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data pre-processing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

There are following steps involved in the data pre-processing:

- **Getting the dataset**
- **Importing libraries**
- **Importing datasets**
- **Finding Missing Data**
- **Encoding Categorical Data**
- **Splitting dataset into training and test set**
- **Feature scaling**

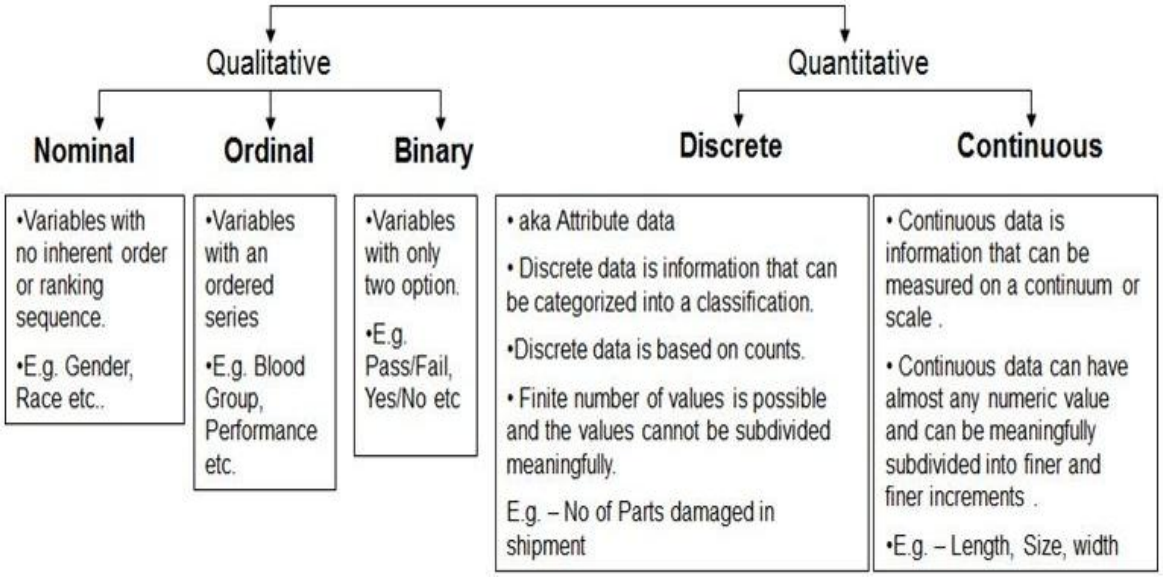
3.4 Data feature engineering: handling categorical data

Data that can be categorized but lacks an inherent hierarchy or order is known as categorical data. A person's gender (male/female), eye colour (blue, green, brown, etc.), type of vehicle they drive (sedan, SUV, truck, etc.), or the kind of fruit they consume (apple, banana, orange, etc.) are examples of categorical data.

Information is represented using two different forms of data:

- **Categorical data**
- **Numeric data**.

Data that may be categorized or grouped together is referred to as categorical data. Men and women fall into the gender category, the colors red, green, and blue fall into the colors category.



Nominal data

Nominal data is categorical data that may be divided into groups, but these groups lack any intrinsic hierarchy or order.

Ordinal data

Ordinal data, on the other hand, describes information that can be categorized and has a distinct order or ranking. Levels of education (high school, bachelor's, master's), levels of work satisfaction (extremely satisfied, satisfied, neutral, unsatisfied, very unsatisfied), and star ratings (1-star, 2-star, 3-star, 4-star, 5-star) are a few examples of ordinal data.

Analysing Categorical Features in Python

There are a few functions in pandas, a popular data analysis library in Python that allow you to quickly analyse categorical data types in your dataset.

A. Value Counts:

value_counts ( ) is a function in the pandas library that returns the frequency of each unique value in a categorical data column. This function is useful when you want to get a quick understanding of the distribution of a categorical variable, such as the most common categories and their frequency.

Syntax: data ['Cut'].value_counts ()

B. Group by

Groupby ( ) is a function in Pandas that allows you to group data by one or more columns and apply aggregate functions such as sum, mean, and count. This function is useful when you want to perform more complex analysis on categorical data, such as computing the average of a numeric variable for each category.

Syntax: data.groupby (by = 'Cut').mean ()

C. Cross tab

Crosstab () is a function in pandas that creates a cross-tabulation table, which shows the frequency distribution of two or more categorical variables. This function is useful when you want to see the relationship between two or more categorical variables, such as how the frequency of one variable is related to another variable.

Syntax: pd.crosstab(index=data['Cut'], columns=data['Color'])

D. Pivot Table

Pivot_table ( ) is a function in Pandas that creates pivot tables, which are similar to cross-tabulation tables but with more flexibility. This function is useful when you want to analyse multiple categorical variables and their relationship to one or more numeric variables. Pivot tables allow you to aggregate data in multiple ways and display the results in a compact form.

Syntax: (pd.pivot_table(data, values='Price', index='Cut', columns='Color', aggfunc=np.mean)

Handing Categorical Features in Python

Categorical data cannot typically be directly handled by machine learning algorithms, as most algorithms are primarily designed to operate with numerical data only. Therefore, before categorical features can be used as inputs to machine learning algorithms, they must be encoded as numerical values.

There are several techniques for encoding categorical features, including one-hot encoding, ordinal encoding, and target encoding. The choice of encoding technique depends on the specific characteristics of the data and the requirements of the machine learning algorithm being used.

1. One-hot encoding:

One hot encoding is a process of representing categorical data as a set of binary values, where each category is mapped to a unique binary value. In this representation, only one bit is set to 1, and the rest are set to 0, hence the name "one hot." This is commonly used in machine learning to convert categorical data into a format that algorithms can process.

id	color
1	red
2	blue
3	green
4	blue

One Hot Encoding

id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

Pandas categorical to numeric

We can use **pd.get_dummies ( )** to categorised in numeric. It is a function in the Pandas library that can be used to perform one-hot encoding on categorical variables in a DataFrame. It takes a DataFrame and returns a new DataFrame with binary columns for each category.

```
Import pandas as pd
# generate df with 1 col and 4 rows
data = {
   "fruit": ["apple", "banana", "orange", "apple"]
}
# show head
df = pd.DataFrame(data)
print(df.head())
```

```
    fruit
0   apple
1  banana
2  orange
3   apple
```

Even though `pandas.get_dummies` is straightforward to use, a more common approach is to use `OneHotEncoder` from the sklearn library, especially when you are doing machine learning tasks.

The primary difference is `pandas.get_dummies` cannot learn encodings; it can only perform one-hot-encoding on the dataset you pass as an input. On the other hand, `sklearn. OneHotEncoder` is a class that can be saved and used to transform other incoming datasets in the future.

```
import pandas as pd
# generate df with 1 col and 4 rows
data = {
   "fruit": ["apple", "banana", "orange", "apple"]
}
# Show head
df = pd.DataFrame(data)
print(df.head( ))
```

```
# apply get_dummies function
df_encoded = pd.get_dummies(df["fruit"])
print(df_encoded .head())
# one-hot-encode using sklearn
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder()
encoded_results = encoder.fit_transform(df).toarray()
print(encoded_results)
```

```
    fruit
0   apple
1  banana
2  orange
3   apple
   apple  banana  orange
0      1       0       0
1      0       1       0
2      0       0       1
3      1       0       0
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]
```

2. Label encoding:

Label encoding is a technique for encoding categorical variables as numeric values, with each category assigned a unique integer. For example, suppose we have a categorical variable "color" with three categories: "red," "green," and "blue." We can encode these categories using label encoding as follows (red: 0, green: 1, blue: 2).

Label encoding can be useful for some machine learning algorithms that require numeric inputs, as it allows categorical data to be represented in a way that the algorithms can understand. However, it's important to keep in mind that label encoding introduces an arbitrary ordering of the categories, which may not necessarily reflect any meaningful relationship between them. In some cases, this can lead to problems in the analysis, especially if the ordering is interpreted as having some kind of ordinal relationship.

Original Data

Team	Points
A	25
A	12
B	15
B	14
B	19
B	23
C	25
C	29

Label Encoded Data

Team	Points
0	25
0	12
1	15
1	14
1	19
1	23
2	25
2	29

Feature Engineering for Categorical Data

Feature engineering is an important step in preparing data for machine learning models. It involves creating new features from the existing features to improve the performance of the models.

There are following ways to perform feature engineering on categorical data:

Creating interaction variables

Interaction variables are new features created by combining two or more existing features. For example, if we have two categorical features, 'Gender' and 'Marital Status,' we can create a new feature, 'Gender-Marital Status,' to capture the interaction between the two features. This can help to capture non-linear relationships between the features and the target variable.

INTERACTION EFFECT

Gender	Marital	Gender_Marital
Male	Married	Male and Married
Male	Unmarried	Male and UnMarried
Female	Unmarried	Female and UnMarried
Male	Married	Male and Married

Binning numerical variables

Binning is the process of dividing continuous numerical variables into discrete bins. This can help to reduce the number of unique values in the feature, which can be beneficial for encoding categorical data. Binning can also help to capture non-linear relationships between the features and the target variable.

BINNING EFFECT

Age	Age_bin
5	0 - 10 year
1	0 - 10 year
21	20 - 30 year
25	20 - 30 year
36	30 - 40 year
39	30 - 40 year
55	>50 year
67	>50 year

3.5 Data validation techniques in python

Data validation is a process in which we checks the data to make sure it meets some rules or restrictions. There various data validation checks we need to do as follow:

- Is of correct data type, for example a number and not a string
- Does not have invalid values, such as provided zip code has a letter
- Is not out of range, such as age given as negative number or there is division by zero
- Meets constraints, for example, given date can only be in future or message does not exceed maximum length
- Is consistent with other data or constraint, for example student test score and student letter grade are consistent

- Is valid, such as given filename is for an existing file
- Is complete, such as making sure all required form fields have data

Types of Validation in Python

There are three types of validation in python, they are:

Type Check: This validation technique in python is used to check the given input data type. For example, int, float, etc.

Length Check: This validation technique in python is used to check the given input string's length.

Range Check: This validation technique in python is used to check if a given number falls in between the two numbers.

Data Validation Tools

1. Cerberus

While working on data, data validation is a crucial task which ensures that the data is cleaned, corrected and is useful. Cerberus is an open source data validation and transformation tool for Python. The library provides powerful and lightweight data validation functionality which can be easily extensible along with custom validation. The Cerberus 1.x versions can be used with Python 2 while version 2.0 and later rely on Python 3 features.

2. Colander

Colander is a Python Library for validating and deserializing data which is obtained via XML, JSON, an HTML form post or any other equally simple data serialisation. It can be said as a good basis for form generation systems, data description systems, and configuration systems. The library has been tested on Python version 2.7 and above and can be used to define a data schema, serialise an arbitrary Python structure to a data structure composed of strings, mappings, and lists and deserialise a data structure composed of strings, mappings, and lists into an arbitrary Python structure after validating the data structure against a data schema.

3. Schema

Schema is a library for validating Python data structures such as those obtained from config-files, forms, external services or command-line parsing, converted from JSON/YAML (or something else) to Python data-types. If the data is valid, Schema. Validate will return the validated data and if the data is invalid, Schema will raise SchemaError exception.

4. Voluptuous

Voluptuous is a Python data validation library. It is primarily intended for validating data coming into Python as JSON, YAML, etc. The library follows mainly three goals which are simplicity, support for complex data structures and providing useful error messages. There are several benefits of this library such as the validators are simple callables, errors are simple exceptions, schemas are basic Python data structures, etc.

5. Valideer

Valideer can be said as the lightweight data validation and adaptation library for Python. It supports both validations (check if a value is valid) and adaptation (convert a valid input to an appropriate output. It is extensible such as the new custom validators and adaptors can be easily defined and registered. The validation schemas can be specified in as declarative and extensible language.

6. Schematics

Schematics is a Python library for data validation which combines types into structures, validate them, and transform the shapes of your data based on simple descriptions. It can also be used in a range of tasks such as design and document specific data structures, convert structures to and from different formats such as JSON or MsgPack, validate API inputs, define message formats for communications protocols, like an RPC, and much more.

Steps to Data Validation

Step 1: Determine data sample

Determine the data to sample. If you have a large volume of data, you will probably want to validate a sample of your data rather than the entire set. You'll need to decide what volume of data to sample, and what error rate is acceptable to ensure the success of your project.

Step 2: Validate the database

Before you move your data, you need to ensure that all the required data is present in your existing database. Determine the number of records and unique IDs, and compare the source and target data fields.

Step 3: Validate the data format

Determine the overall health of the data and the changes that will be required of the source data to match the schema in the target. Then search for incongruent or incomplete counts, duplicate data, incorrect formats, and null field values.

Methods for Data Validation

You can perform data validation in one of the following ways:

A. **Scripting:** Data validation is commonly performed using a scripting language such as Python to write scripts for the validation process. For example, you can create an XML file with source and target database names, table names, and columns to compare. The Python script can then take the XML as an input and process the results. However, this can be very time intensive, as you must write the scripts and verify the results by hand.

B. **Enterprise tools:** Enterprise tools are available to perform data validation. For example, FME data validation tools can validate and repair data. Enterprise tools have the benefit of being more stable and secure, but can require infrastructure and are costlier than open source options.

C. **Open source tools:** Open source options are cost-effective, and if they are cloud-based, can also save you money on infrastructure costs. But they still require a level of knowledge and hand-coding to be able to use effectively. Some open source tools are SourceForge and OpenRefine.

Challenges in Data Validation

There may be the various key challenge in data validation:

- Validating the database can be challenging because data may be distributed in multiple databases across your organization. The data may be siloed, or it may be outdated.
- Validating the data format can be an extremely time-consuming process, especially if you have large databases and you intend to perform the validation manually. However, sampling the data for validation can help to reduce the time needed.

3.6 Data feature engineering: removing columns and rows from raw data

- **Python Pandas Drop Function**

  Pandas **drop** is a function in Python pandas used to drop the rows or columns of the dataset. This function is often used in data cleaning.

  Note: axis = 0 is referred as rows and axis = 1 is referred as columns.